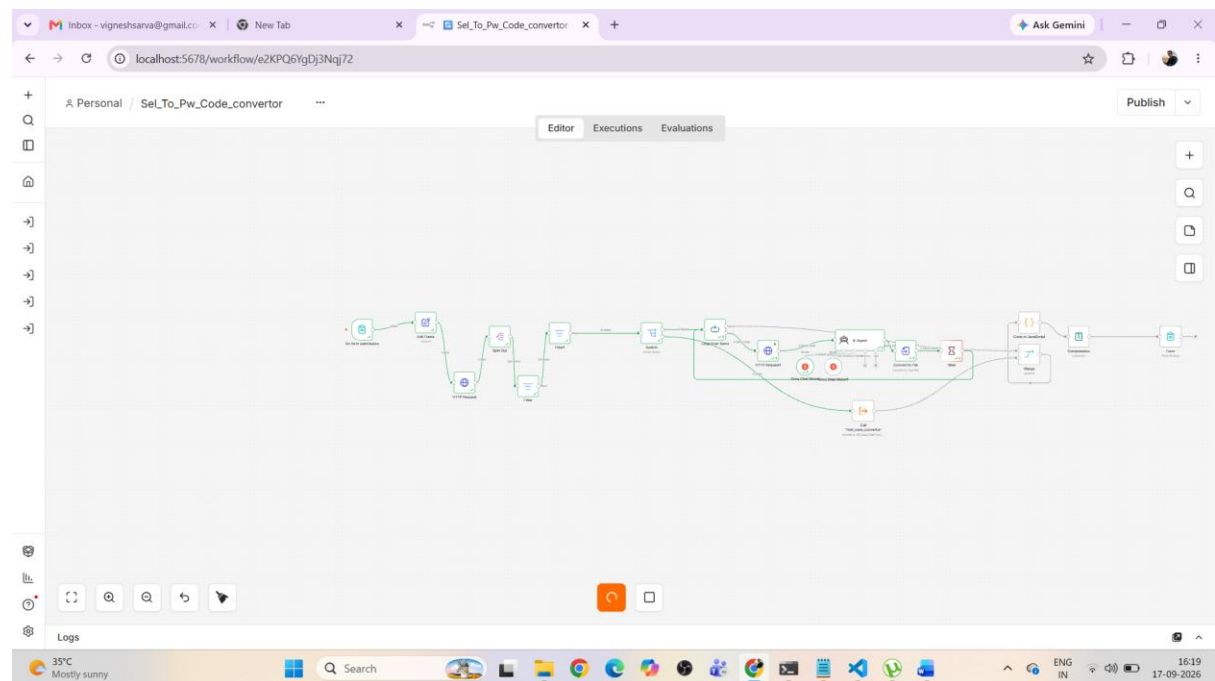


The screenshot shows the n8n web interface with a workflow titled "SelTo_Pw_Code_converter". The workflow is published and is visible in the "Overview" tab. The workflow consists of the following nodes:

- Load Spreadsheet
- Email Read
- Split Out
- Filter
- Filter
- Filter
- Switch
- Email Read
- AI Agent
- Export to File
- Merge
- Email Send
- Email Send

The workflow is titled "SelTo_Pw_Code_converter" and is published. The interface includes a sidebar with "Overview", "Favorites", "Templates", "Insights", "Help", and "Settings". The top bar shows the n8n logo and the workflow title. The bottom bar shows the system status and the time.



Call 'Test_case_converter'

INPUT

Schema Table JSON

Parameters Settings

Source Database

Workflow From list Test_case_converter

This sub-workflow will consume all input data passed to it. You can define specific expected input in the sub-workflow's trigger.

Mode Run once for each item

"Run once for each item" is deprecated and will be removed in a future version. To run the sub-workflow once per item, add a "Loop Over Items" node before this node and use "Run once with all items".

Options + Add option

OUTPUT

Executing node...

Convert to File

INPUT

Schema Table JSON

Parameters Settings

Operation Convert to Text File

Text Input Field output

Put Output File in Field data

The name of the output binary field to put the file in

Options + Add option

File Name {{ \$('Loop Over Items').item.json.path.split('pages/')[1].replace('.java','.txt') }}.txt

WorkTypeGroupPage.txt

OUTPUT

Run 5 of 5 (1 item)

data

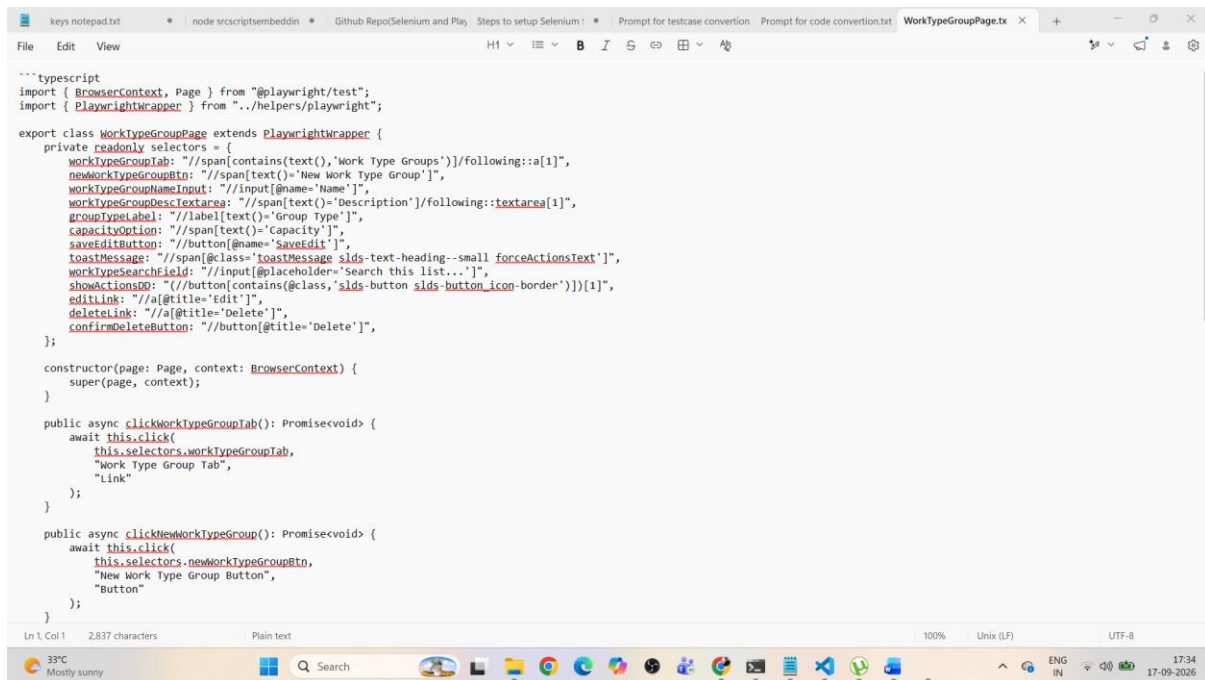
File Name: WorkTypeGroupPage.txt

File Extension: txt

Mime Type: text/plain

File Size: 2.84 kB

View Download



```
````typescript
import { BrowserContext, Page } from "@playwright/test";
import { PlaywrightWrapper } from "../helpers/playwright";

export class WorkTypeGroupPage extends PlaywrightWrapper {
 private readonly selectors = {
 workTypeGroupTab: "//span[contains(text(),'Work Type Groups')]/following::a[1]",
 newWorkTypeGroupBtn: "//span[text()='New Work Type Group']",
 workTypeGroupNameInput: "//input[@name='Name']",
 workTypeGroupDescTextarea: "//span[text()='Description']/following::textarea[1]",
 groupTypeLabel: "//label[text()='Group Type']",
 capacityOption: "//span[text()='Capacity']",
 saveEditButton: "//button[@name='SaveEdit']",
 toastMessage: "//span[@class='toastMessage slds-text-heading--small forceActionsText']",
 workTypeSearchField: "//input[@placeholder='Search this list...']",
 showActionsDD: "(//button[contains(@class,'slds-button slds-button_icon-border')])[1]",
 editLink: "//a[@title='Edit']",
 deleteLink: "//a[@title='Delete']",
 confirmDeleteButton: "//button[@title='Delete']",
 };

 constructor(page: Page, context: BrowserContext) {
 super(page, context);
 }

 public async clickWorkTypeGroupTab(): Promise<void> {
 await this.click(
 this.selectors.workTypeGroupTab,
 "Work Type Group Tab",
 "Link"
);
 }

 public async clickNewWorkTypeGroup(): Promise<void> {
 await this.click(
 this.selectors.newWorkTypeGroupBtn,
 "New Work Type Group Button",
 "Button"
);
 }
}
```

```
````typescript
```

```
import { BrowserContext, Page } from "@playwright/test";
```

```
import { PlaywrightWrapper } from "../helpers/playwright";
```

```
export class WorkTypeGroupPage extends PlaywrightWrapper {
```

```
  private readonly selectors = {
```

```
    workTypeGroupTab: "//span[contains(text(),'Work Type Groups')]/following::a[1]",
```

```
    newWorkTypeGroupBtn: "//span[text()='New Work Type Group']",
```

```
    workTypeGroupNameInput: "//input[@name='Name']",
```

```
    workTypeGroupDescTextarea: "//span[text()='Description']/following::textarea[1]",
```

```
    groupTypeLabel: "//label[text()='Group Type']",
```

```
    capacityOption: "//span[text()='Capacity']",
```

```
    saveEditButton: "//button[@name='SaveEdit']",
```

```
    toastMessage: "//span[@class='toastMessage slds-text-heading--small  
forceActionsText']",
```

```
    workTypeSearchField: "//input[@placeholder='Search this list...']",
```

```
    showActionsDD: "(//button[contains(@class,'slds-button slds-button_icon-  
border')])[1]",
```

```
editLink: "//a[@title='Edit']",  
deleteLink: "//a[@title='Delete']",  
confirmDeleteButton: "//button[@title='Delete']",  
};
```

```
constructor(page: Page, context: BrowserContext) {  
    super(page, context);  
}
```

```
public async clickWorkTypeGroupTab(): Promise<void> {  
    await this.click(  
        this.selectors.workTypeGroupTab,  
        "Work Type Group Tab",  
        "Link"  
    );  
}
```

```
public async clickNewWorkTypeGroup(): Promise<void> {  
    await this.click(  
        this.selectors.newWorkTypeGroupBtn,  
        "New Work Type Group Button",  
        "Button"  
    );  
}
```

```
public async enterWorkTypeGroupName(gName: string): Promise<void> {  
    await this.type(  
        this.selectors.workTypeGroupNameInput,
```

```
        "Work Type Group Name Input",  
        gName  
    );  
}
```

```
public async enterWorkTypeDesc(desc: string): Promise<void> {  
    await this.type(  
        this.selectors.workTypeGroupDescTextarea,  
        "Work Type Group Description Textarea",  
        desc  
    );  
}
```

```
public async selectGroupType(): Promise<void> {  
    await this.click(  
        this.selectors.groupTypeLabel,  
        "Group Type Label",  
        "Label"  
    );  
    await this.click(  
        this.selectors.capacityOption,  
        "Capacity Option",  
        "Option"  
    );  
}
```

```
public async clickWorkTypeSaveButton(): Promise<void> {  
    await this.click(  
        this.selectors.workTypeSaveButton,
```

```

this.selectors.saveEditButton,

"Save Button",

"Button"

);
}

```

```

public async verifyWorkTypeGroups(text: string): Promise<void> {

    await this.validateElementVisibility(

        this.selectors.toastMessage,

        "Toast Message"

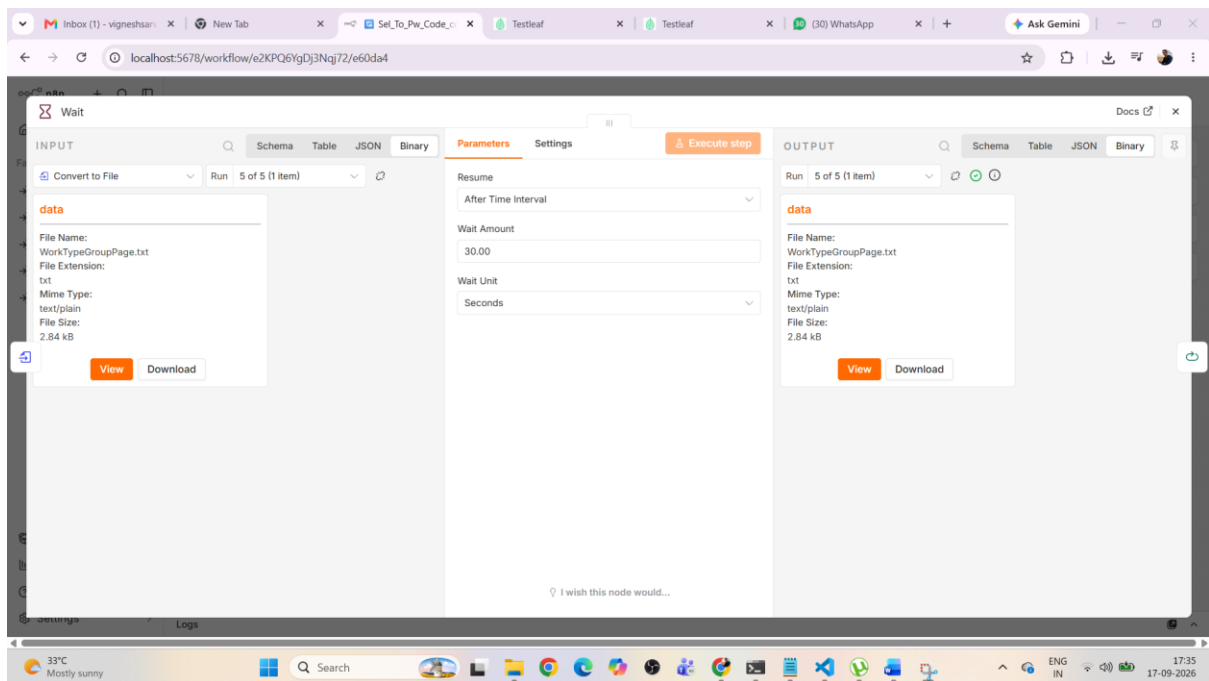
    );
}

```

```

public async enterWorkTypeSearchField

```



Call 'Test_case_converter'

INPUT

Switch 5 items

path CRM_25-main/CRM_25-main/src/main/java/com/salesforce/testcases/TC001_CreateNewCase.java

mode 100644

type blob

sha ca06129b50c2dd5dd4da5c95a616ac337d621221

size 1489

url https://api.github.com/repos/Harrish-sdet/CRM/git/blobs/ca06129b50c2dd5dd4da5c95a616ac337d621221

Filter1 10 items

Filter 20 items

Split Out 103 items

HTTP Request 1 item

Edit Fields 1 item

On form submission 1 item

Variables and context

Parameters

Source Database

Workflow From list Test_case_converter

This sub-workflow will consume all input data passed to it. You can define specific expected input in the sub-workflow's trigger.

Mode Run once for each item

Options + Add option

EXECUTE STEP

OUTPUT

5 items, 5 sub-executions

TC004_DeleteIndividuals.txt

File Extension: txt

Mime Type: text/plain

File Size: 1.85 kB

View Download

5

data

File Name: TC005_CreateWorkType.txt

File Extension: txt

Mime Type: text/plain

File Size: 0 B

View Download

```
...typescript
import { test } from "../customFixtures/salesForceFixture";
import { FakerData } from "../helpers/fakerUtils";

/* ----- Test Metadata ----- */
test.info().annotations.push(
  { type: 'TestCase', description: 'TC_001 Create New Case' },
  { type: 'TestDescription', description: 'Creates and verifies a new Salesforce Case using data from Excel' },
  { type: 'Author', description: 'Hari' },
  { type: 'Category', description: 'Regression' }
);

/* ----- Fixture Storage State ----- */
// If a login state is already captured, use it to skip manual login.
// Otherwise, the SalesforceLogin fixture will perform the login steps.
test.use({ storageState: "logins/salesforceLogin.json" });

/* ----- Test Data ----- */
const searchField = "Cases";
const salutation = "Mr.";
const status = "New";
const origin = "Web";
const subject = FakerData.getRandomSentence();
const description = FakerData.getParagraph();

/* ----- Test Implementation ----- */
test('TC_001 Create New Case', async ({
  SalesforceLogin,
  SalesforceHome,
  SalesforceCase,
  SalesforceContact,
}) => {
  /* Login to Salesforce */
  await SalesforceLogin.salesforceLogin("ADMINLOGIN");

  /* Navigate to Cases via App Launcher */
  await SalesforceHome.appLauncher();
  await SalesforceHome.viewAll();
  await SalesforceHome.searchApp(searchField);
  await SalesforceHome.selectApp("Cases");
});
```

Convert to File

INPUT

AI Agent1

The fields below come from the last successful execution. Execute node to refresh them.

output

HTTP Request1

Loop Over Items

Wait

When Executed by Another Workflow

Variables and context

Parameters

Operation Convert to Text File

Text Input Field output

Put Output File in Field data

The name of the output binary field to put the file in

Options + Add option

File Name

fx {{ \$('Loop Over Items').item.json.path.split('testcases/')[1].replace('.java','') }}.txt

EXECUTE STEP

OUTPUT

No output data

Execute step

or set mock data

Loop Over Items

Docs | x

INPUT

SchemaTableJSON

Wait

Execute previous nodes
to view input data

When Executed by Another Workflow

Convert to File

AI Agent1

HTTP Request1

Variables and context

Preview

Parameters

Settings

Execute step

You may not need this node — n8n nodes automatically run once for each input item. [More info](#)

Batch Size

1

Options

+ Add option

I wish this node would...

OUTPUT

SchemaTableJSON

No nodes connected to the 'loop' output of this node

I→

No output data

Execute step

to view output data

AI Agent1

Docs | x

INPUT

SchemaTableJSON

HTTP Request1

The fields below come from the last successful execution. [Execute node](#) to refresh them.

data

Loop Over Items

Wait

When Executed by Another Workflow

Convert to File

Variables and context

Preview

Parameters

Settings

Execute step

Tip: Get a feel for agents with our quick tutorial or see an example of how this node works

Source for Prompt (User Message)

Define below

Prompt (User Message)

P - PERSONA
You are a Principal Test Automation Architect who is an expert in both legacy Java Selenium (TestNG, POM, DataProvider) and modern Playwright/TestCafe automation frameworks. You

Require Specific Output Format

Enable Fallback Model

Connect an additional language model on the canvas to use it as a fallback if the main model fails

Options

+ Add Option

Chat Model * Fallback Model * Memory Tool

OUTPUT

SchemaTableJSON

I→

No output data

Execute step

or set mock data